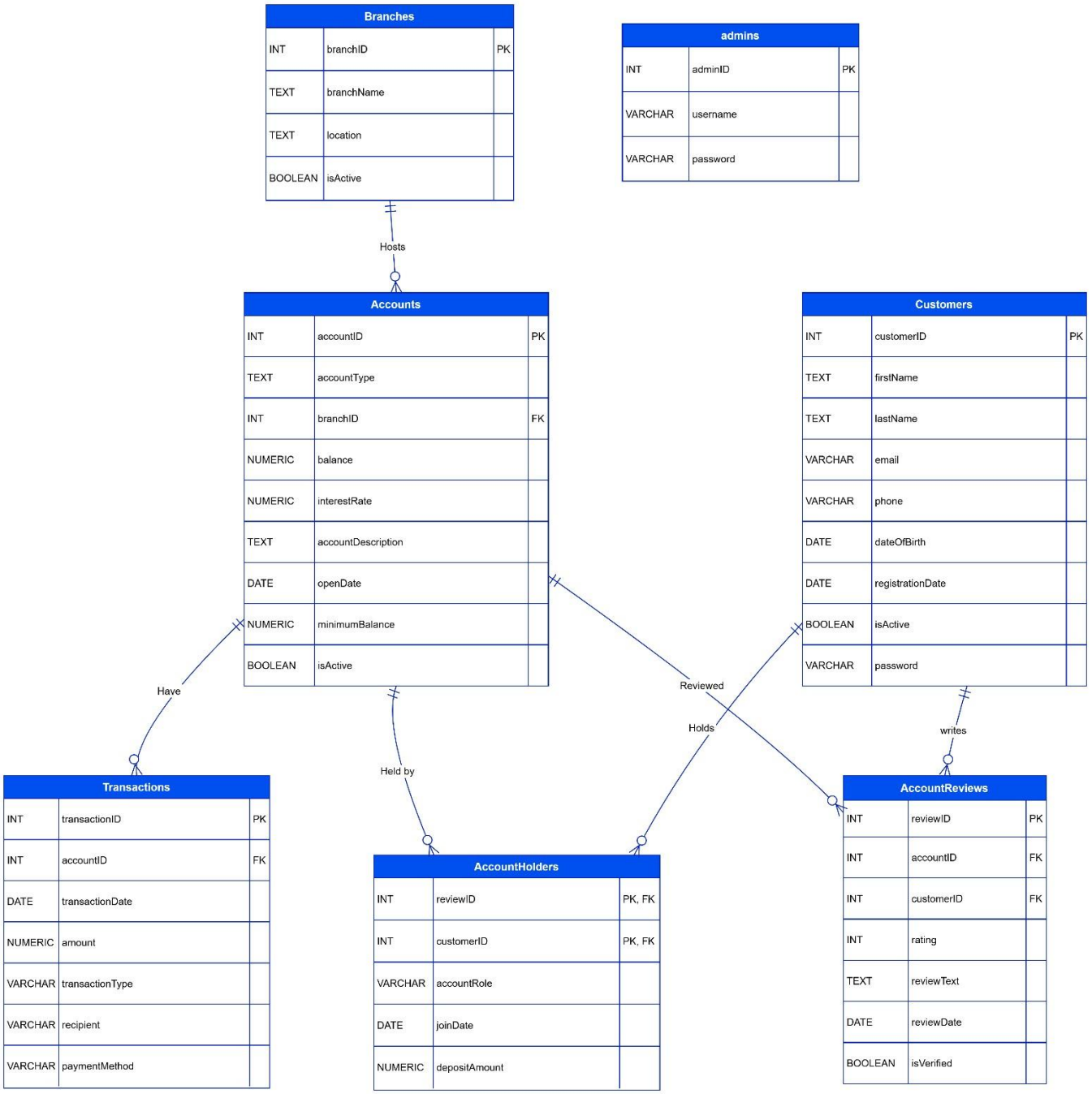
Banking Application Armin Mujcinovic 4-24-2026

Database Design:

ER Diagram:



Schema Explanation:

Branches: This entity stores the physical location data for bank branches.

Accounts: This entity stores the account information of a user. Information included in this entity include financial data such as Checking's and Saving accounts. In these accounts, branchID, balance, interest rate, account description, opening date, minimum balance, and if account is active are all stored.

Customers: This entity stores customer information such as customerID, first name, last name, email address, phone number, date of birth, date of registration, status of the customer, and their password. Customers log in to the bank app using their email address and password unf123.

AccountHolders: This entity serves as a table that joins accounts and customers together in order to tie financial data together. Accounts and customers need to be joined in order to pull financial records and account reviews.

Transactions: This entity provides a comprehensive list of all the transactions made by an account.

AccountReviews: This entity provides a way to store customer reviews and ratings. Reviews are approved and verified by only the administrators of the banking system.

Admins: This table stores the credentials for administrators using the bank app. To provide role based access this is a crucial aspect of the schema. Admins log in with username admin1 and password admin123.

Relationships:

Branches ➏ Hosts ➏ Accounts (1:N)

Accounts ➏ Have ➏ Transactions (1:N)

Accounts ➏ Reviewed ➏ AccountReviews (1:N)

Customer ➏ Writes ➏ AccountReviews (1:N)

Via accountHolders customers ➏ hold ➏ Accounts (M:N)

Modifications:

Apart from the trigger function and stored procedures that were required of the assignment to modify the database, I have added two major components to this project. The first being creates an Admins table that stores administrators' credential information such as their username and password which is universally defined as username = admin1 and password = admin123. The second component that I added was adding a password section to the Customers table. The password for all customers is unf123. These two critical design decisions were made in order to provide a secure and accurate login to the application and a role-based decision making process when it comes to executing certain functions in the application. Screenshots below show what I have changed in the database.

```
CREATE TABLE Customers(
customerID INT PRIMARY KEY,
firstName TEXT,
lastName TEXT,
email VARCHAR(150) UNIQUE,
phone VARCHAR(20),
dateOfBirth DATE,
registrationDate DATE,
isActive BOOLEAN
);
ALTER TABLE Customers ADD COLUMN password VARCHAR(255) DEFAULT 'unf123';
```

```
CREATE TABLE admins (
adminID INT PRIMARY KEY,
username varchar(50) UNIQUE,
password varchar(250)
);

INSERT INTO Admins (adminID, username, password) VALUES
(1, 'admin1', 'admin123');
SELECT * FROM ADMINS;
```

Implementation Details:

Technology Stack:

Language: Java Version 17 for the backend logic

Database: PostgreSQL

Driver: JDBC -> PostgreSQL Driver

Tools: pgAdmin4 for database management and InelliJ IDEA for the application

Key Design Decisions:

Role-Based Access: I choose to implement a role access-based decision-making system for my banking app application. This was done to differentiate what end users can do within the application, and it had to follow strict protocols. For example, if customer A wants to selftransfer money from one of his accounts to another account, the application must check that customer A owns both accounts. In the application users were given a userRole which was compared in the functions with userRole.equalsIgnoreCase("admin") or an else to differentiate a customer and a admin.

ACID Properties: ACID properties were essential when users would make transactions. With these properties I was able to make sure that if a user attempted to transfer more money from their account to another account the transaction would be rolled back. This was also important for admins to cancel and reverse false transactions, so no money was lost between accounts. I utilized conn.setAutoCommit() so the withdrawals from one account and a deposit to another account to ensure a single atomic unit.

Database Logic vs Application Logic: I placed complex balance updates and data validation methods into my database instead of my application. I noticed when I was writing the code in my application that it was long and uneasy to read so I decided to use PostgreSQL Triggers and Stored Functions to ensure data integrity and make the process quicker.

I also decided to get rid of inserting sample data function. The function named public static void insertSampleData(Connection conn). This caused me errors and I had plenty of insertion functions for each module that required inserting new data into the tables and this confused me.

Challenges Faced & Solutions

SQL Syntax errors when inserting dynamic ID increments:

Since the database schema did not use a SERIAL keyword for the ID's of customers, accounts, reviews, etc. I decided to create my own functions in my application. I found it troubling to go back and redo the whole create tables and decided to create my own version of incrementing IDs when inserting new records. Each function is clearly labeled on type of ID being made for example generateAccountID. Each function has a static variable for example in reviews private static final int *REVIEW_MIN_ID* = 7000 and this is where the IDs start from the database. Then increments are made from selecting the max ID already in the database or if no data start from 7000 so the first reviewID would be 7001.

Handling "Self" recipient string in the integer-based AccountID column:

I solved this challenge by revising the database trigger to ignore the recipient column when the transaction type is a standard Deposit or Withdrawal.

Transferring Java variables into Database:

I faced an issue with asking the end user for input for data information such as balance and date. In java balance would typically be a double but in my database, I had NUMERIC which was a problem. So, to fix these issues I type casted after variables in my database like this balance::NUMERIC and in my application when using select queries. Also, for transferring strings such as deposit or self-transfer I used ILIKE in order for my database to read the data being inserted correctly.

Trouble with connecting the database to application:

I first visited office hours to get my connection going however I found more issues that I might have missed. I noticed as I was trying to pull data from the database nothing was coming that database appeared empty, and I knew for sure that this was not the case. To fix this issue I added a executable statement to set my search path to the same path I had in my database. For example, stmt.execute("SET search_path TO Banking");.

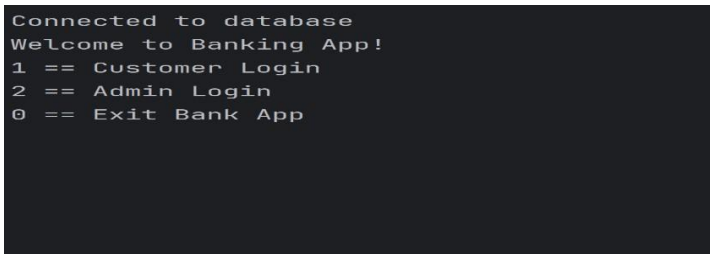
User Guide:

To connect to the database make sure URL, username, password, and search path are set:

```java
public class BankApp {

    private static final String DB_URL = "jdbc:postgresql://localhost:5432/Banking_DB";
    private static final String USER = "postgres";
    private static final String PASS = "your_password";


    public static Connection getConnection() {
        try {
            // TODO: Establish connection
            Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
            Statement stmt = conn.createStatement();
            stmt.execute("SET search_path TO Banking");
            System.out.println("Connected to database");
            return conn;
        } catch (SQLException e) {
            System.out.println("Connection error: " + e.getMessage());
            return null;
        }
    }
```



When connected to the database user can select admin or customer login.

Customer Login Page username is email and password is always <u>unf123 (TEST ONLY PWD)</u>.
Customer and
Admin have this next screen of bank app features but once clicked will have different functions to choose from based on role (Customer or Admin).

CUSTOMER LOGIN CREDITIALS:

Username ❼ Customers Email Address

Password ❼ unf123

ADMIN LOGIN CREDNTIALS:

Username ❼ admin1

Password ❼ admin123

```
1
Please enter username / email:
john.smith@email.com
Please enter password:
unf123
Welcome back Customer #5001

||| WELCOME TO THE BANK APP |||
1. Branch Management Module
2. Account Management Module
3. Customer Management Module
4. Transaction Processing Module
5. Review Management Module
0. Logout
```

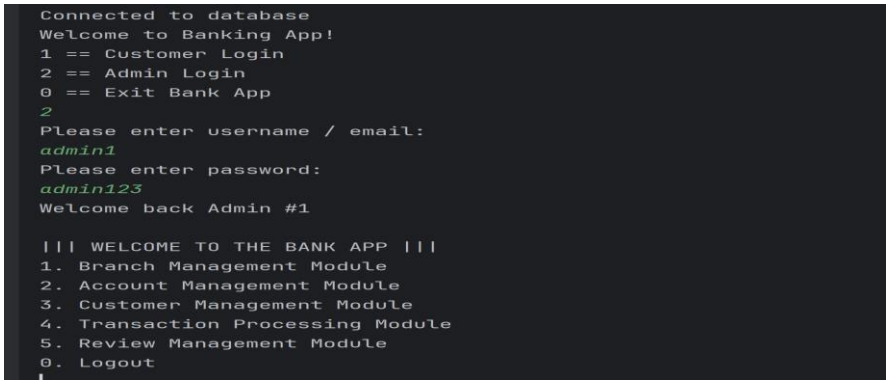
For example, here's how a customer role menu for customer module looks like:

```
||| WELCOME TO THE BANK APP |||
1. Branch Management Module
2. Account Management Module
3. Customer Management Module
4. Transaction Processing Module
5. Review Management Module
0. Logout
3
WELCOME TO CUSTOMER MODULE
1. Register as a new customer
2. Update personal information (phone, email)
3. View personal profile
4. Deactivate own account
0. Return
0
```

And here's how a Admin sees the customer module admins have more features typically

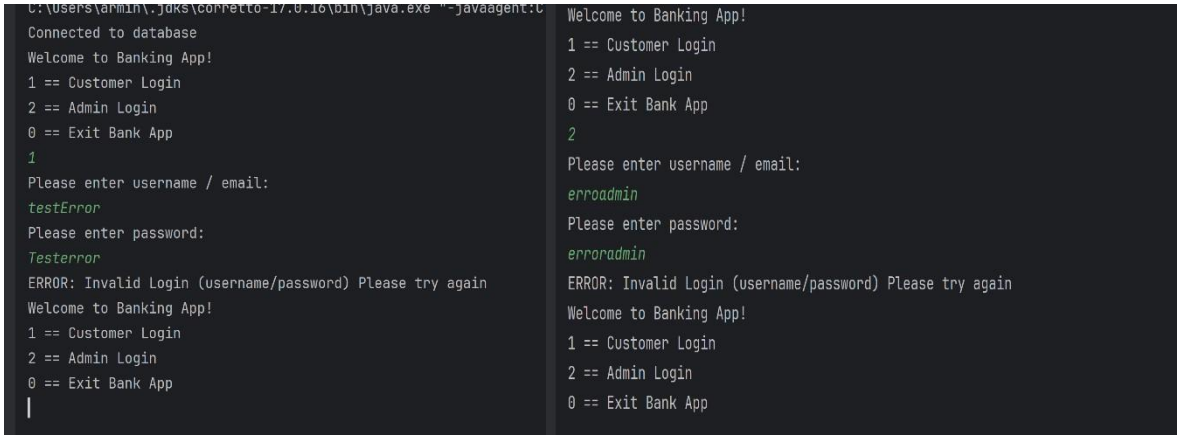
```
||| WELCOME TO THE BANK APP |||
1. Branch Management Module
2. Account Management Module
3. Customer Management Module
4. Transaction Processing Module
5. Review Management Module
0. Logout
3
WELCOME TO CUSTOMER MODULE
1. List all customers with registration date
2. View a customer's full account history
3. View a customer's transaction history across all accounts
4. View a customer's review history
5. Add new customer
0. Return
```

Admin Login:



More features:

Invalid credentials for admin and customers both users are redirected to login again:



Test Cases:

Question 1: Prevent Overdraft (Balance Validation)

```
Welcome back Customer #5001

||| WELCOME TO THE BANK APP |||
1. Branch Management Module
2. Account Management Module
3. Customer Management Module
4. Transaction Processing Module
5. Review Management Module
0. Logout
4
WELCOME TO TRANSACTION MODULE
1. Deposit / Withdrawal
2. Transfer
0. Return
1
Account ID: 1001
Amount: 3000
Type (Deposit/Withdrawal): withdrawal
Balance: 2500.0
ERROR: Account #1001 has insufficient funds Available balance: $2500.0
1. Deposit / Withdrawal
2. Transfer
0. Return
```

Question 2: Automatic Balance Update on Transaction

```
1. Deposit / Withdrawal
2. Transfer
0. Return
1
Account ID: 1001
Amount: 1000
Type (Deposit/Withdrawal): Deposit
Deposit transaction complete.. NEW BALANCE: $3500.0
1. Deposit / Withdrawal
2. Transfer
0. Return
1
Account ID: 1001
Amount: 1000
Type (Deposit/Withdrawal): withdrawal
withdrawal transaction complete.. NEW BALANCE: $2500.0
```